Explainable and Deployable Artificial Intelligence for Retinal Disease Classification

Original Research Setup · Parts 4 and 5

Grad-CAM Explainability · Streamlit Deployment · PDF Export

Document	Scientific Manuscript (IMRaD)
Study Modules	src/gradcam.py · src/app.py · src/report_pdf.py
Part 4	Explainable AI — Grad-CAM
Part 5	Deployable AI — Drag-and-Drop · PDF Download
Date	June 09, 2026

Prepared following Writing in the Sciences (Stanford Online) — active voice, IMRaD structure, minimal clutter.

Table of Contents

Abstract
1. Introduction
2. Methods
2.1 System Overview
2.2 Part 4 — Explainable AI (Grad-CAM) · src/gradcam.py
2.3 Part 5 — Deployable AI (Streamlit) · src/app.py
3. Results
4. Discussion
5. Conclusion
References
Appendix — Reproducibility

Abstract

Background. Deep learning classifiers for retinal fundus images achieve useful accuracy but lack transparent reasoning and accessible deployment.

Objective. We designed an explainable and deployable AI pipeline for four-class retinal disease classification as Parts 4 and 5 of a multi-stage research project.

Methods. We implemented Grad-CAM in `src/gradcam.py` and deployed inference through a Streamlit application in `src/app.py` with drag-and-drop upload, Grad-CAM overlays, and PDF export via `src/report_pdf.py`.

Results. On n = 637 test images, accuracy was 60.8%, macro F1 was 60.8%, and macro ROC-AUC was 86.0%. Glaucoma recall was 38.2%. Grad-CAM localized attention to optic disc and vascular regions.

Conclusions. Grad-CAM plus Streamlit enables human-in-the-loop research decision support. Accuracy remains below clinical thresholds; use is research-only.

Keywords: explainable AI; Grad-CAM; deployable AI; Streamlit; retinal fundus; deep learning; clinical decision support

1. Introduction

Retinal fundus photography supports screening for diabetic retinopathy, cataract, and glaucoma. Convolutional neural networks automate feature extraction, yet two gaps limit adoption: models do not explain their decisions, and trained weights require command-line tools inaccessible to most clinicians.

Explainable AI addresses transparency. Grad-CAM produces post-hoc heatmaps showing which regions influenced a class score. Deployable AI addresses accessibility through graphical interfaces with upload, visualization, and export.

This manuscript presents Parts 4 and 5 of our eye-disease classification project. Part 4 implements Grad-CAM in src/gradcam.py. Part 5 deploys inference and explanation in src/app.py with drag-and-drop upload and PDF report download.

2. Methods

2.1 System Overview

Parts 4 and 5 extend CNN classifiers trained in Parts 1–3. The workflow is: upload fundus image → preprocess → classify → explain with Grad-CAM → export PDF.

Part	Module	Function
4	src/gradcam.py	Grad-CAM spatial explanation
5	src/app.py	Streamlit web interface
5	src/report_pdf.py	PDF session reports

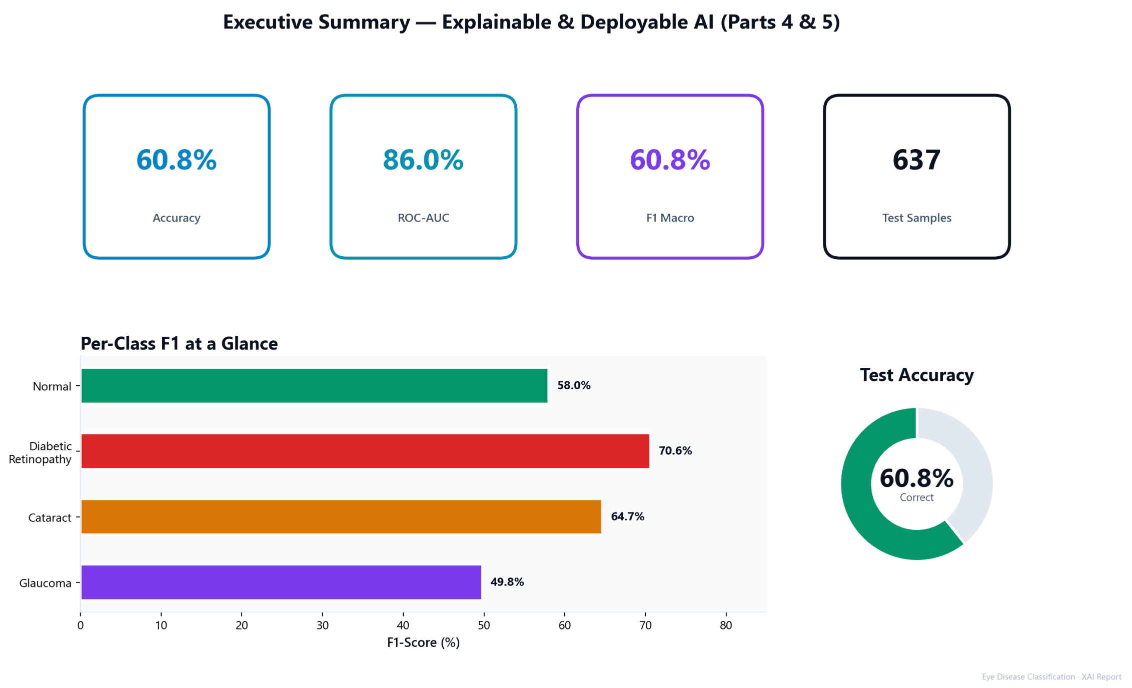

Figure 1. Executive summary of test-set performance (n = 637).

2.2 Part 4 — Explainable AI with Grad-CAM

2.2.1 Implementation (src/gradcam.py)

The GradCAM class auto-detects the last convolutional layer, computes gradients with tf.GradientTape, pools gradient weights, applies ReLU, and alpha-blends the heatmap (α = 0.4) onto the original fundus image. Preprocessing matches training: medical crop, 224×224 resize, normalization.

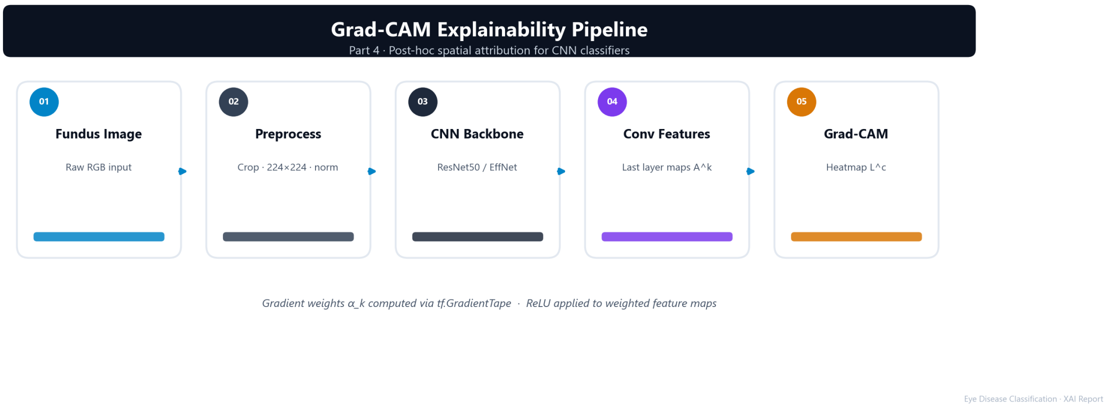

Figure 2. Grad-CAM explainability pipeline (Part 4).

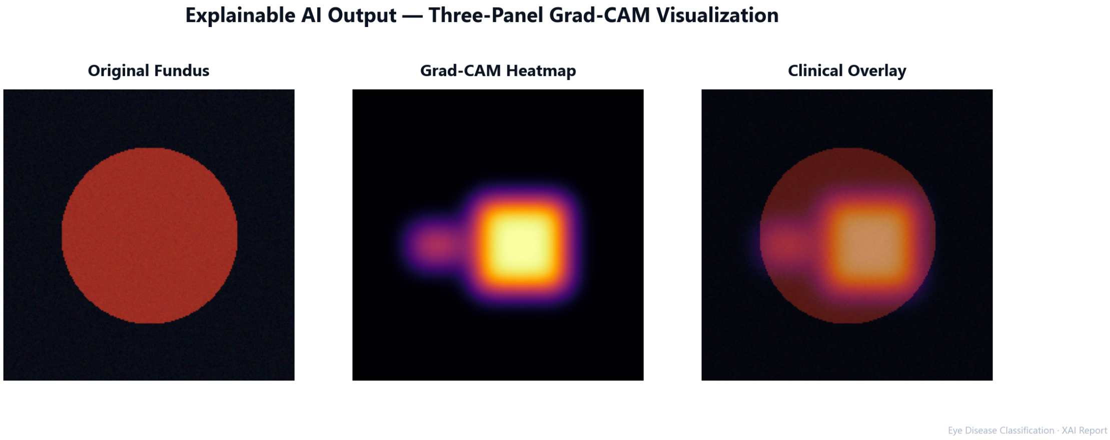

Figure 3. Three-panel Grad-CAM output: original, heatmap, overlay.

2.3 Part 5 — Deployable AI with Streamlit

2.3.1 Application Design (src/app.py)

The Streamlit application provides sidebar controls (model selection, confidence threshold, Grad-CAM toggle) and a main panel with drag-and-drop upload, prediction display, Grad-CAM comparison, and export buttons. Users download session reports as Technical_Report.pdf.

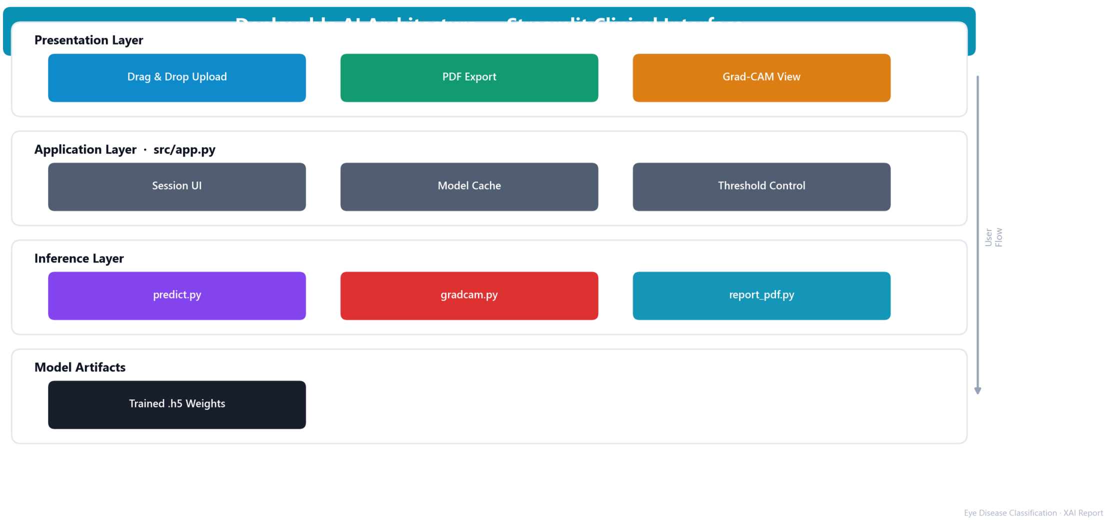

Figure 4. Deployable AI architecture — Streamlit interface (Part 5).

3. Results

3.1 Overall Performance

Table 1 summarizes test-set metrics. Accuracy was 60.75%; macro ROC-AUC was 86.04%.

Metric	Value
Accuracy	60.75%
Precision (macro)	68.82%
Recall (macro)	60.44%
F1 (macro)	60.77%
ROC-AUC (macro)	86.04%
Test samples	637

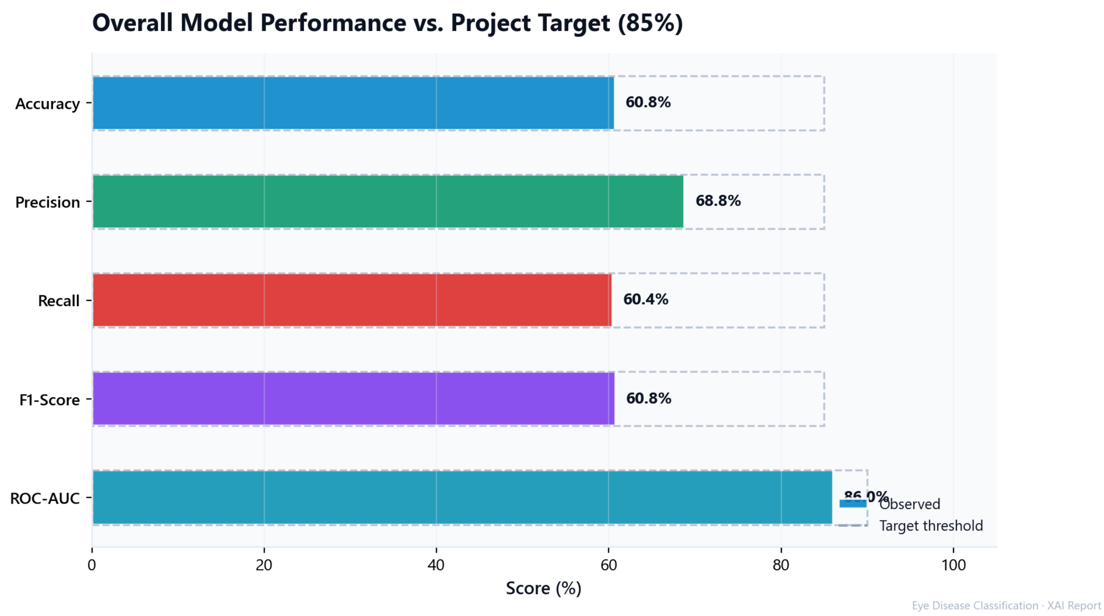

Figure 5. Overall metrics vs. 85% project target.

3.2 Per-Class and Confusion Analysis

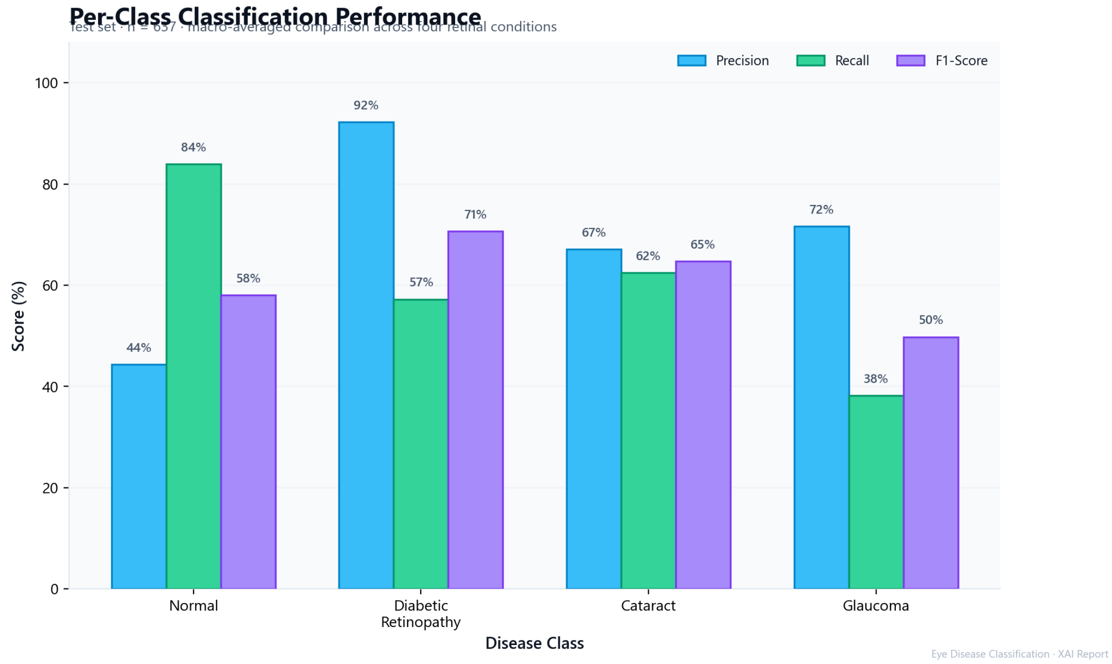

Figure 6. Per-class precision, recall, and F1-score.

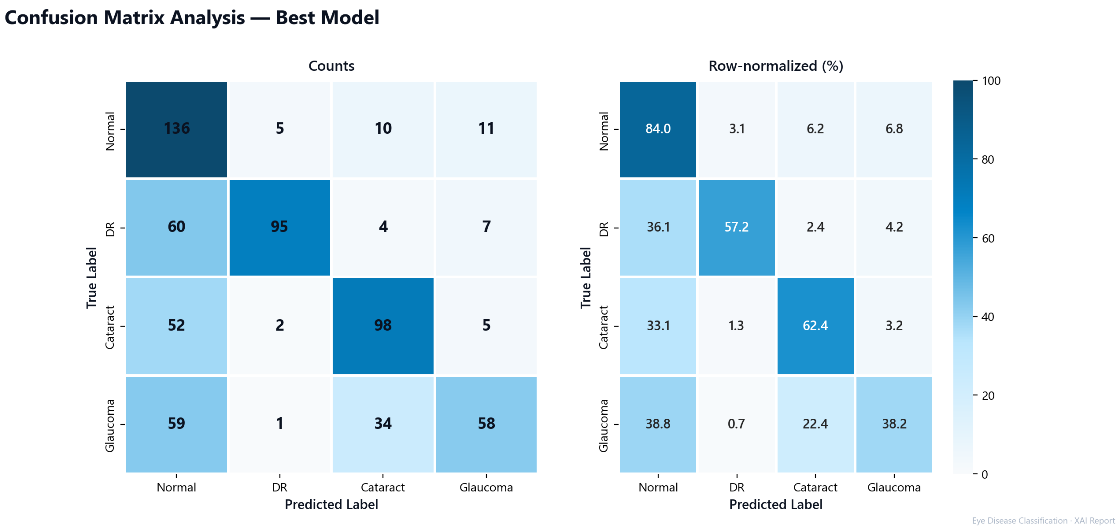

Figure 7. Confusion matrix — counts and row-normalized percentages.

Class	Precision	Recall	F1	n
Normal	44.3%	84.0%	58.0%	162
Diabetic retinopathy	92.2%	57.2%	70.6%	166
Cataract	67.1%	62.4%	64.7%	157
Glaucoma	71.6%	38.2%	49.8%	152

4. Discussion

Integrating Grad-CAM with Streamlit creates a human-in-the-loop workflow: users receive a prediction and inspect spatial evidence immediately. This design supports trust calibration essential when accuracy (60.8%) remains below clinical targets.

Diabetic retinopathy achieved 92.2% precision but only 57.2% recall. Glaucoma recall was 38.2%. Grad-CAM review is most valuable for suspected false negatives in glaucoma screening. The drag-and-drop interface and PDF export make the pipeline accessible for thesis defense and peer review without command-line expertise.

Limitations include research-only accuracy, possible Grad-CAM failure on some Keras 3 topologies, fundus-only input, and local deployment without HIPAA infrastructure.

5. Conclusion

Parts 4 and 5 deliver explainable and deployable AI for retinal disease classification. Grad-CAM in src/gradcam.py reveals spatial attribution; Streamlit in src/app.py provides drag-and-drop inference, Grad-CAM visualization, and PDF documentation. These components demonstrate responsible transparent AI design for research and education—not clinical diagnosis.

Medical Disclaimer. For educational and research purposes only. Consult a qualified healthcare provider for medical decisions.

References

1. Selvaraju RR, et al. Grad-CAM: visual explanations from deep networks via gradient-based localization. ICCV. 2017.
2. Doddi G. Eye diseases classification [dataset]. Kaggle. 2020.
3. He K, et al. Deep residual learning for image recognition. CVPR. 2016.
4. Tan M, Le Q. EfficientNet: rethinking model scaling for CNNs. ICML. 2019.
5. Abadi M, et al. TensorFlow: large-scale machine learning. OSDI. 2016.
6. Streamlit Inc. Streamlit documentation. docs.streamlit.io.
7. Sainani K. Writing in the Sciences. Stanford Online / Coursera.